LAYERED BANKING SYSTEM: SERVICE-REPOSITORY ARCHITECTURE (JAVA)

TWINKLE SRIVASTWA, CSE(IOTCSBT)

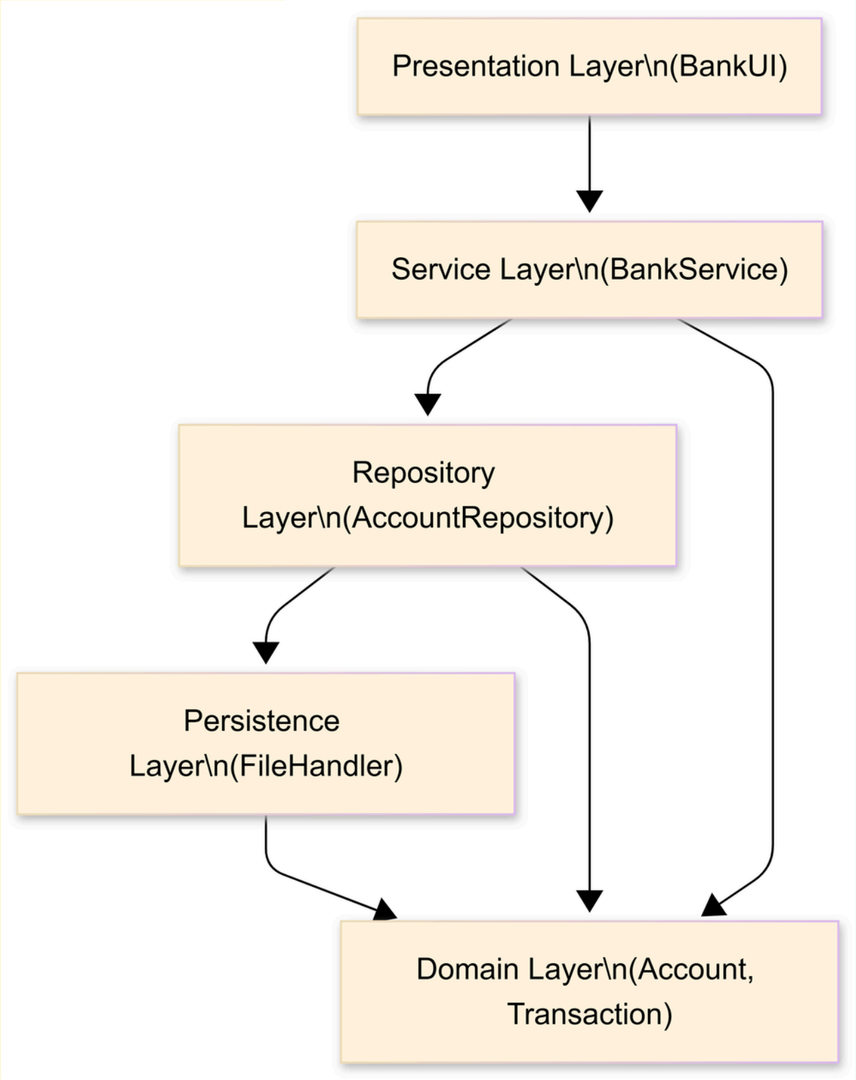
PROBLEM STATEMENT:

MONOLITHIC BANKING CODE → TIGHTLY COUPLED LOGIC → HARD TO MAINTAIN/EXTEND/DEBUG
NEED MODULAR, LAYERED SYSTEM WITH VALIDATION & PERSISTENCE

TARGET USERS:

BANKING CUSTOMERS & STAFF
TRANSACTION MANAGEMENT & ACCOUNT OPERATIONS

SYSTEM DESIGN

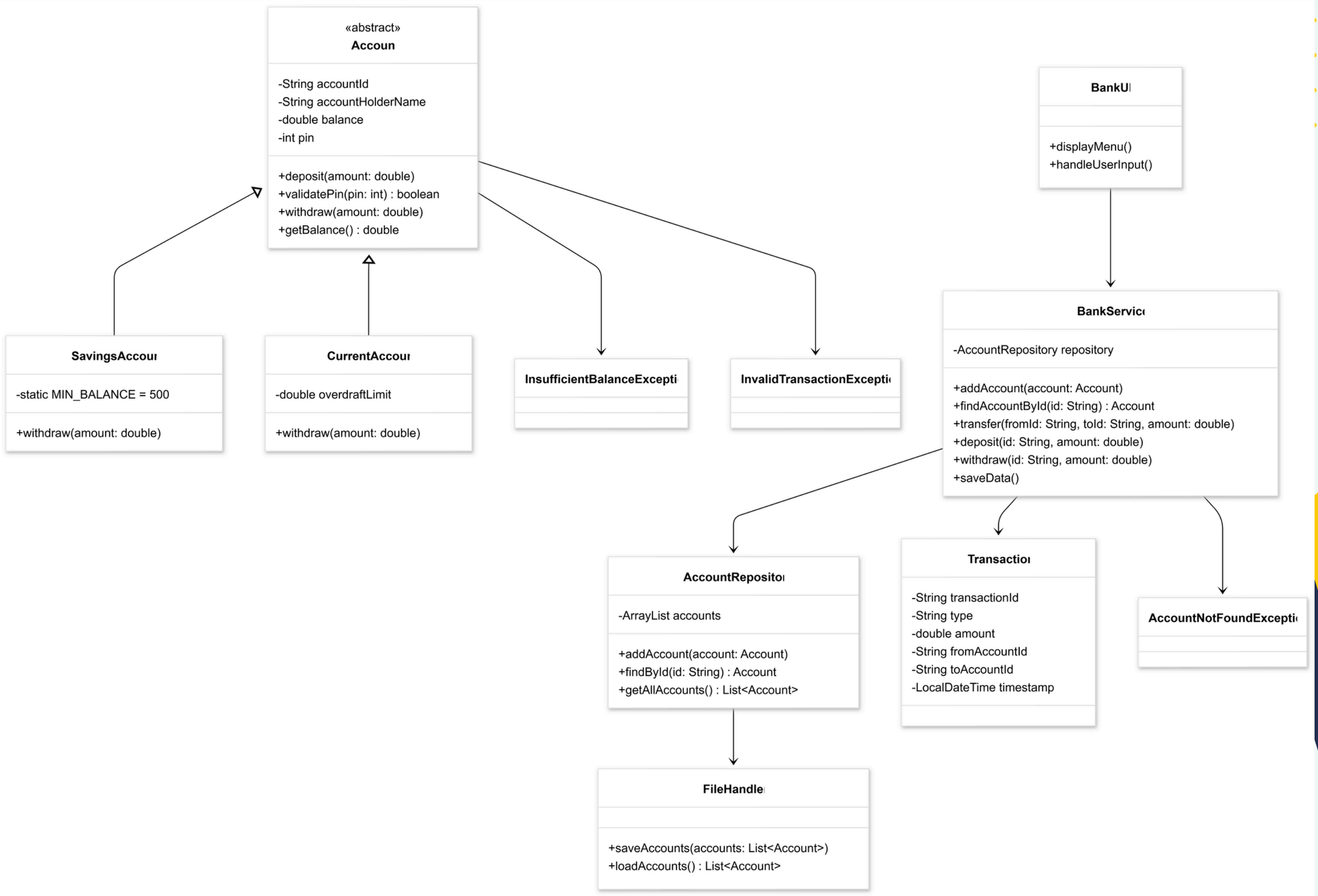


KEY CLASSES

- **ACCOUNT (ABSTRACT):** ACCOUNTID, BALANCE, WITHDRAW()
- **SAVINGSACCOUNT**: MIN BALANCE ₹500
- **CURRENTACCOUNT:** OVERDRAFT SUPPORT
- **TRANSACTION:** TIMESTAMP, TYPE, FROM/TO
- **BANKSERVICE:** DEPOSITS, WITHDRAWALS, TRANSFERS, INPUT VALIDATION
- **ACCOUNTREPOSITORY:** CRUD ABSTRACTION
- **FILEHANDLER:** CSV PERSISTENCE

PACKAGE STRUCTURE

- BANKING.DOMAIN – ACCOUNT, TRANSACTION
- BANKING.SERVICE – BANKSERVICE
- BANKING.REPOSITORY – REPOSITORY INTERFACE
- BANKING.PERSISTENCE – FILEHANDLER
- BANKING.EXCEPTION – CUSTOM EXCEPTIONS
- BANKING.UI – SWING/CONSOLE


«abstract»
Accoun
-String accountId
-String accountHolderName
-double balance
-int pin
+deposit(amount: double)
+validatePin(pin: int) : boolean
+withdraw(amount: double)
+getBalance() : double
BankUI
+displayMenu()
+handleUserInput()
SavingsAccou
-static MIN_BALANCE = 500
+withdraw(amount: double)
CurrentAccou
-double overdraftLimit
+withdraw(amount: double)
InsufficientBalanceExcepti
InvalidTransactionExcepti
BankServic
-AccountRepository repository
+addAccount(account: Account)
+findAccountById(id: String) : Account
+transfer(fromId: String, toId: String, amount: double)
+deposit(id: String, amount: double)
+withdraw(id: String, amount: double)
+saveData()
AccountRepositor
-ArrayList accounts
+addAccount(account: Account)
+findById(id: String) : Account
+getAllAccounts() : List<Account>
Transactio
-String transactionId
-String type
-double amount
-String fromAccountId
-String toAccountId
-LocalDateTime timestamp
AccountNotFoundExcepti
FileHandle
+saveAccounts(accounts: List<Account>)
+loadAccounts() : List<Account>

OOPS DESIGN

CONCEPT WHERE	EXAMPLE PURPOSE	BENEFIT
ABSTRACTION	ACCOUNT.WITHDRAW()	HIDES IMPLEMENTATION; ENFORCES UNIFORM INTERFACE
INHERITANCE	SAVINGSACCOUNT / CURRENTACCOUNT	REUSE BASE ATTRIBUTES & OVERRIDE RULES
POLYMORPHISM	ACCOUNT ACC = NEW SAVINGSACCOUNT()	RUNTIME DYNAMIC BEHAVIOR
ENCAPSULATION	PRIVATE BALANCE, PIN	PROTECT SENSITIVE DATA; CONTROLLED ACCESS
DESIGN PATTERN	SERVICE-REPOSITORY	DECOUPLE BUSINESS LOGIC FROM PERSISTENCE

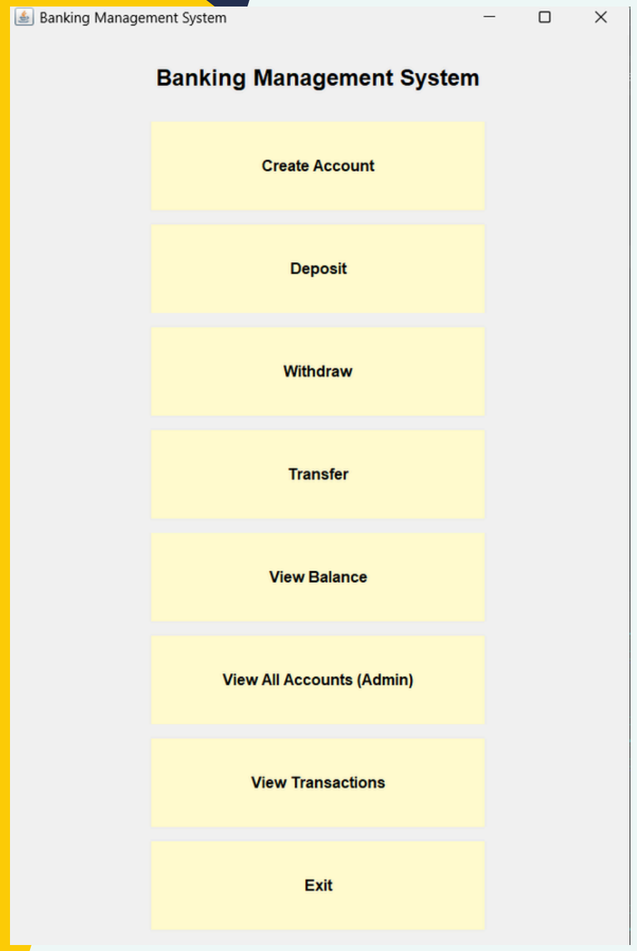
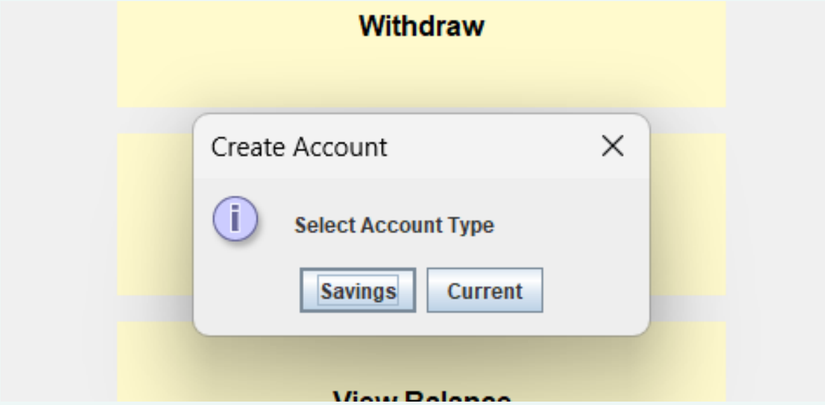
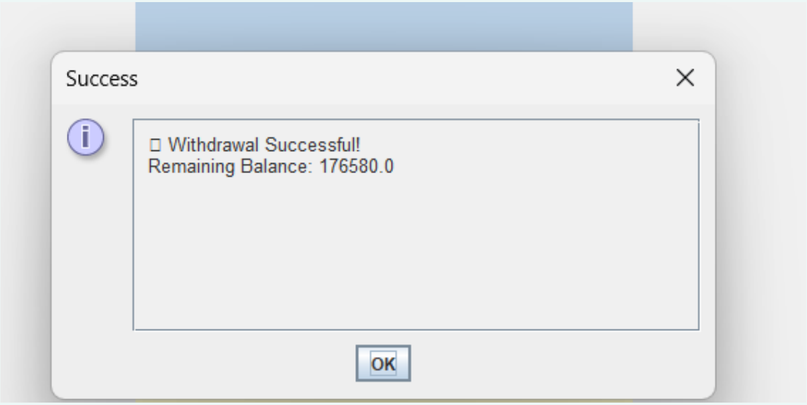
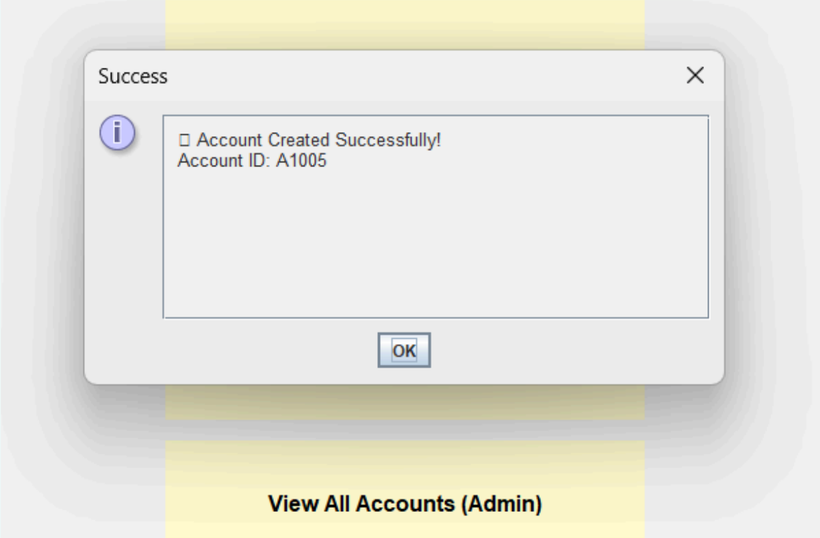
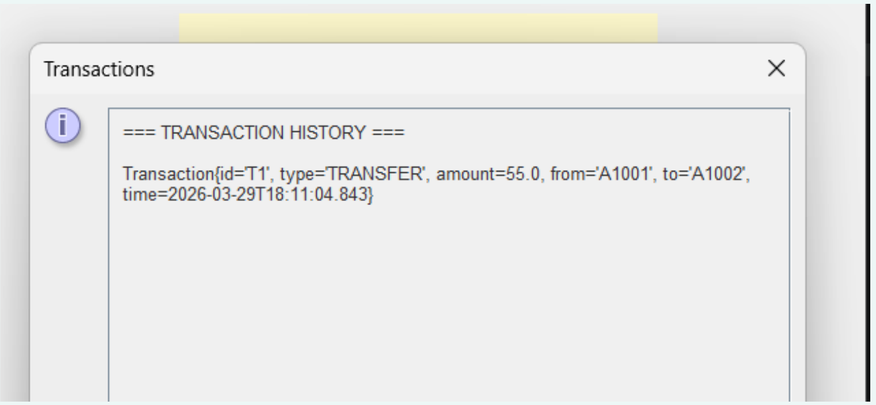
TECHNICAL HIGHLIGHT

REPOSITORY LAYER - DATA ABSTRACTION

- Abstracts data access from business logic
- Provides CRUD operations: findById(), save(), update(), delete()
- Enables swapping storage backend (CSV → Database) without touching BankService
- Central point for all account-related persistence
- Maintains modular, maintainable, testable architecture

ATOMIC TRANSACTION FLOW IN TRANSFERS

```
from.withdraw(amount);
to.deposit(amount);
transactions.add(new Transaction("TRANSFER", amount, fromId, toId));
```

WITHDRAW → DEPOSIT → LOG → CONSISTENCY & TRACEABILITY

LIVE DEMONSTRATION